

Clean Code

Material de Estudo

Análise e Desenvolvimento de Sistemas | PUC Goiás

1. Introdução

Código limpo é código que pode ser lido, compreendido e modificado por qualquer desenvolvedor do time com o menor esforço possível. A definição parece simples, mas suas implicações são profundas: ela desloca o foco do ato de fazer o programa funcionar para o ato de comunicar intenção de forma precisa e duradoura.

O conceito foi sistematizado por Robert C. Martin no livro *Clean Code: A Handbook of Agile Software Craftsmanship* (2008), obra que consolidou práticas dispersas da comunidade em um conjunto coerente de princípios. Martin parte de uma premissa central: a maior parte do tempo de um desenvolvedor é gasta lendo código, não escrevendo. Estudos da própria indústria estimam que a relação entre tempo de leitura e escrita é de aproximadamente dez para um. Escrever código difícil de ler é, portanto, uma decisão que prejudica todo trabalho futuro sobre aquela base.

"Qualquer idiota pode escrever código que um computador entende. Bons programadores escrevem código que humanos entendem." (Martin Fowler, Refactoring, 1999)

Este material apresenta os principais princípios do Clean Code com exemplos práticos em Java. O objetivo não é fornecer regras mecânicas a seguir cegamente, mas desenvolver o julgamento necessário para escrever código que resista ao tempo e à mudança.

2. Nomenclatura: a primeira forma de documentação

O nome de uma variável, função, classe ou pacote é a primeira coisa que qualquer leitor processa. Um nome bem escolhido elimina a necessidade de comentário; um nome mal escolhido cria ambiguidade que nenhum comentário consegue desfazer completamente.

2.1 Nomes que revelam intenção

O nome deve responder por que algo existe, o que faz e como é usado. Se for necessário um comentário para explicar o nome, o nome está errado.

Problemático

```
int d; // dias desde criação

List<int[]> lista1;

for (int[] x : lista1) {
    if (x[0] == 4) {
        resultado.add(x);
    }
}
```

Limpo

```
int diasDesdeUltimaModificacao;

List<int[]> celulasAtivas;

for (int[] celula : celulasAtivas) {
    if (celula[STATUS] == ATIVA) {
        resultado.add(celula);
    }
}
```

2.2 Evitar desinformação

Nomes que sugerem algo diferente do que representam são piores do que nomes genéricos. Chamar um conjunto de contas de `accountList` quando o tipo subjacente não é uma `List` sugere estrutura errada ao leitor. Prefira `accounts` ou `groupOfAccounts`.

Variações sutis entre nomes longos são especialmente traiçoeiras. A distinção entre `XYZControllerForEfficientHandlingOfStrings` e `XYZControllerForEfficientStorageOfStrings` é quase invisível em uma leitura rápida.

2.3 Distinções significativas

Se dois nomes existem, eles devem significar coisas diferentes. Nomes como `a1`, `a2`, `a3` ou `info` e `data` em vez de termos específicos do domínio são não-informativos. Da mesma forma, prefixos como `The` ou `An` (`theCustomer`, `aCustomer`) não acrescentam distinção real.

2.4 Pronunciabilidade e buscabilidade

Nomes devem ser pronunciáveis, o que facilita discussões em equipe. Nomes de variáveis de uma letra só são aceitáveis em escopos muito pequenos, como contadores em laços curtos. Fora desse contexto, dificultam buscas no código e reduzem a legibilidade.

Problemático

```
class DtaRcrdl02 {  
    private Date genymdhms;  
    private Date modymdhms;  
    private final String pszqint = "102";  
}
```

Limpo

```
class Cliente {  
    private Date dataCriacao;  
    private Date dataModificacao;  
    private final String codigoRegistro =  
        "102";  
}
```

2.5 Convenções para classes e métodos

- Classes devem ter nomes substantivos ou sintagmas nominais: Cliente, ContaBancaria, ProcessadorDePagamento. Nunca verbos.
- Métodos devem ter nomes verbais ou sintagmas verbais: calcularTotal(), salvarCliente(), buscarPorId(). Acessores e modificadores seguem a convenção getX, setX, isX.
- Constantes devem ser escritas em UPPER_SNAKE_CASE: TAXA_JUROS_MENSAL, MAX_TENTATIVAS.

3. Funções

Funções são a unidade fundamental de organização do código. Os princípios de Clean Code para funções são mais prescritivos do que os de nomenclatura, porque violações nesta área têm consequências diretas sobre manutenibilidade e testabilidade.

3.1 Tamanho reduzido e responsabilidade única

Uma função deve fazer uma coisa, fazê-la bem e fazer apenas ela. A regra prática é que se for possível extrair outra função de uma função existente com um nome que não seja simplesmente uma reformulação de sua implementação, é sinal de que a função original está fazendo mais de uma coisa.

Funções curtas são preferíveis. Martin sugere entre cinco e vinte linhas como orientação. O argumento não é estético, mas funcional: funções curtas são mais fáceis de nomear, testar e compreender isoladamente.

Problemático

```
// Função fazendo três coisas:  
// validar, calcular e persistir  
public void processarPedido(Pedido p) {
```

Limpo

```
public void processarPedido(Pedido p) {  
    validarPedido(p);  
    p.setTotal(calcularTotal(p));  
}
```

```

if (p.getItems().isEmpty()) {
    throw new RuntimeException("Vazio");
}
BigDecimal total = BigDecimal.ZERO;
for (Item i : p.getItems()) {
    total = total.add(i.getPreco()
        .multiply(BigDecimal.valueOf(
            i.getQuantidade())));
}
p.setTotal(total);
pedidoRepository.save(p);
}

```

```

pedidoRepository.save(p);
}

private void validarPedido(Pedido p) {
    if (p.getItems().isEmpty())
        throw new PedidoVazioException();
}

private BigDecimal calcularTotal(Pedido
p) {
    return p.getItems().stream()
        .map(i -> i.getPreco().multiply(
            BigDecimal.valueOf(i.getQuantidade())))
        .reduce(BigDecimal.ZERO,
            BigDecimal::add);
}

```

3.2 Argumentos

O número ideal de argumentos de uma função é zero (niladic). Um argumento (monadic) é aceitável; dois (dyadic) requerem atenção; três (triadic) devem ser evitados sempre que possível. Acima disso, a função provavelmente está assumindo responsabilidades demais ou os argumentos deveriam ser agrupados em um objeto.

Argumentos booleanos são um sinal de alerta: indicam que a função faz duas coisas diferentes dependendo do valor passado. A solução é dividir em duas funções com nomes distintos.

Problemático

```

// Argumento booleano: a função
// faz duas coisas diferentes
public void renderizar(
    boolean ehPaginaCompleta) {
    if (ehPaginaCompleta) {
        // renderiza página completa
    } else {
        // renderiza trecho
    }
}

```

Limpo

```

public void renderizarPaginaCompleta() {
    // renderiza página completa
}

public void renderizarTrecho() {
    // renderiza trecho
}

```

3.3 Sem efeitos colaterais

Uma função que promete fazer X e secretamente faz Y também é enganosa e perigosa. Efeitos colaterais criam acoplamento temporal: o chamador precisa saber em que ordem chamar as funções para evitar comportamentos inesperados. Funções puras, que dependem apenas de seus argumentos e não alteram estado global, são inerentemente mais previsíveis e testáveis.

3.4 Separação entre comandos e consultas

Uma função deve ou fazer algo (modificar estado) ou responder algo (retornar informação), nunca ambos. Esse princípio, conhecido como Command-Query Separation (CQS), evita confusões sobre o que a função faz ao ser chamada.

Problemático	Limpo
<pre>// Faz algo E retorna resultado // ao mesmo tempo public boolean definirAtributo(String nome, Object valor) { if (atributoExiste(nome)) { atributos.put(nome, valor); return true; } return false; }</pre>	<pre>// Separado: consulta + comando public boolean atributoExiste(String nome) { return atributos.containsKey(nome); } public void definirAtributo(String nome, Object valor) { atributos.put(nome, valor); }</pre>

4. Comentários

Comentários são, na maioria das vezes, um fracasso de expressão: indicam que o código não conseguiu comunicar sua intenção por si só. A regra de Martin é dura e vale a pena enunciar com exatidão: o melhor comentário é aquele que você encontrou uma maneira de não precisar escrever.

Isso não significa que comentários são proibidos. Existem comentários legítimos e valiosos. O problema é que comentários desnecessários tendem a envelhecer sem que o código seja atualizado junto, tornando-se uma fonte de desinformação mais perigosa do que a ausência de qualquer explicação.

4.1 Comentários que compensam código ruim

O impulso de comentar frequentemente surge diante de código confuso. A resposta correta é limpar o código, não adicionar comentário. Um módulo repleto de comentários e cheio de advertências sobre o que fazer e o que não fazer pode, na maioria dos casos, ser substituído por código mais expressivo que dispense essas instruções.

4.2 Comentários legítimos

- Comentários de licença no início de arquivos, quando exigidos pela organização.
- Explicações de intenção quando a escolha de implementação não é óbvia, especialmente em otimizações ou algoritmos não triviais.
- Esclarecimentos sobre comportamentos de APIs externas que não podem ser alterados.
- Avisos de consequências de modificações não óbvias.

- Comentários TODO como notas de trabalho pendente, desde que monitorados.
- Javadoc em APIs públicas de bibliotecas.

Problemático

```
// Retorna os dias trabalhados
// no mês atual pelo funcionário
public int getDtm() {
    return dtm;
}
```

Limpo

```
// Nenhum comentário necessário:
// o nome já comunica tudo
public int getDiasTrabalhados() {
    return diasTrabalhados;
}
```

4.3 Comentários que devem ser evitados

- Comentários que apenas repetem o que o código já diz (ruído).
- Código comentado: use controle de versão; código morto polui e confunde.
- Comentários de posição (seções separadas por linhas de asteriscos ou hifens).
- Comentários de autoria e data de modificação, já capturados pelo sistema de controle de versão.

5. Formatação

Formatação é comunicação. O estilo com que o código é organizado visualmente transmite estrutura, intenção e contexto antes mesmo que uma linha seja lida com atenção. Uma equipe deve definir um conjunto de regras de formatação e segui-las consistentemente; em Java, ferramentas como Checkstyle, PMD e o próprio formatador do IDE automatizam grande parte desse trabalho.

5.1 Formatação vertical

Arquivos menores são geralmente mais fáceis de entender do que arquivos grandes. Como referência, a maioria dos arquivos Java de um projeto bem estruturado deve ter entre 100 e 500 linhas; arquivos com mais de 1.000 linhas costumam sinalizar violação de responsabilidade única.

Conceitos relacionados devem estar verticalmente próximos. Variáveis de instância ficam no topo da classe. Funções que se chamam mutuamente devem ser posicionadas próximas umas das outras. A função chamadora deve aparecer antes da chamada, de modo que o código possa ser lido de cima para baixo como uma narrativa.

5.2 Formatação horizontal

Cada linha deve ser curta o suficiente para não exigir rolagem horizontal. Uma referência comum é o limite de 120 caracteres, embora 80 ainda seja adotado em muitos projetos. Espaços em torno de operadores e após vírgulas melhoram a legibilidade sem custo algum.

```
// Organização vertical de uma classe
public class ContaBancaria {

    // (1) Constantes de classe
    private static final BigDecimal LIMITE_PADRAO = new BigDecimal("1000.00");

    // (2) Variáveis de instância
    private final String numeroConta;
    private BigDecimal saldo;

    // (3) Construtor
    public ContaBancaria(String numeroConta) {
        this.numeroConta = numeroConta;
        this.saldo = BigDecimal.ZERO;
    }

    // (4) Métodos públicos
    public void depositar(BigDecimal valor) {
        validarValorPositivo(valor);
        this.saldo = this.saldo.add(valor);
    }

    public void sacar(BigDecimal valor) {
        validarValorPositivo(valor);
        validarSaldoSuficiente(valor);
        this.saldo = this.saldo.subtract(valor);
    }

    // (5) Métodos privados (auxiliares dos públicos acima)
    private void validarValorPositivo(BigDecimal valor) {
        if (valor.compareTo(BigDecimal.ZERO) <= 0)
            throw new IllegalArgumentException("Valor deve ser positivo.");
    }

    private void validarSaldoSuficiente(BigDecimal valor) {
        if (saldo.compareTo(valor) < 0)
            throw new SaldoInsuficienteException(numeroConta);
    }
}
```

6. Objetos e estruturas de dados

Clean Code distingue com precisão dois tipos de entidades que frequentemente são confundidas: objetos e estruturas de dados. Compreender essa distinção é essencial para fazer escolhas corretas de design.

Objetos escondem seus dados atrás de abstrações e expõem funções que operam sobre esses dados. Estruturas de dados expõem seus dados diretamente e não possuem funções significativas. Não existe versão híbrida coerente: um objeto que expõe getters e setters para todos os atributos é, na prática, uma estrutura de dados com overhead de método.

6.1 Lei de Demeter

A Lei de Demeter (também conhecida como princípio do menor conhecimento) estabelece que um módulo não deve conhecer os detalhes internos dos objetos que manipula. Formalmente: um método *f* de uma classe *C* deve chamar apenas métodos de *C*, de objetos criados por *f*, de objetos passados como argumentos a *f* e de objetos guardados em variáveis de instância de *C*.

Problemático

```
// Viola a Lei de Demeter:  
// navega pela estrutura interna  
String cidade = funcionario  
    .getEndereco()  
    .getCidade()  
    .getNome();
```

Limpo

```
// Expõe o que o cliente precisa  
// sem revelar a estrutura  
String cidade = funcionario  
    .getCidadeResidencia();  
  
// Funcionario encapsula a navegação
```

6.2 Data Transfer Objects (DTOs)

DTOs são estruturas de dados legítimas: seu propósito é transportar informação entre camadas sem lógica associada. Em Java, são comuns como registros de banco de dados, payloads de API ou formulários de interface. Não devem ser confundidos com entidades de domínio, que são objetos com comportamento.

7. Tratamento de erros

Código de tratamento de erros que obscurece a lógica principal não é código limpo. O tratamento de erros é importante, mas não deve dominar o fluxo de leitura do código.

7.1 Prefira exceções a códigos de retorno

Verificar o valor de retorno de cada chamada polui o código chamador e mistura lógica de controle com lógica de negócio. Exceções permitem separar o caminho feliz do tratamento de erro de forma clara.

Problemático

```
// Código de erro misturado
// com lógica de negócio
int resultado =
processador.processar(cmd);
if (resultado ==
ERRO_DISPOSITIVO_OCUPADO) {
    logger.error("Dispositivo ocupado");
    // tratar...
} else if (resultado == ERRO_TIMEOUT) {
    logger.error("Timeout");
    // tratar...
}
```

Limpo

```
// Separação clara entre
// caminho feliz e exceções
try {
    processador.processar(cmd);
} catch (DispositivoOcupadoException e)
{
    logger.error("Dispositivo ocupado",
e);
} catch (TimeoutException e) {
    logger.error("Timeout", e);
}
```

7.2 Não retorne null

Retornar null força cada chamador a verificar null, e um null esquecido provoca NullPointerException em local distante de onde o problema se originou. As alternativas em Java são: retornar `Optional<T>` quando a ausência é um resultado válido, lançar exceção quando a ausência é um erro, ou retornar coleção vazia quando o resultado é uma lista.

Problemático

```
public Cliente buscarCliente(String id)
{
    // retorna null se não encontrar
    return clienteMap.get(id);
}

// Chamador obrigado a checar
Cliente c = buscarCliente(id);
if (c != null) {
    c.processar();
}
```

Limpo

```
public Optional<Cliente> buscarCliente(
    String id) {
    return Optional.ofNullable(
        clienteMap.get(id));
}

// Intenção explícita no tipo
buscarCliente(id)
    .ifPresent(Cliente::processar);
```

7.3 Não passe null

Assim como retornar null é problemático, passar null como argumento para métodos é uma prática que deve ser evitada. Exceções não verificadas são preferíveis a verificações de null espalhadas pelo código.

8. Testes limpos

Testes sujos são piores do que nenhum teste. Código de teste que é difícil de manter tende a ser abandonado conforme o sistema cresce, anulando os benefícios de ter testes. Clean Code aplica os mesmos critérios de legibilidade ao código de teste que ao código de produção.

8.1 Um assert por teste (como orientação)

Cada método de teste deve verificar um único comportamento. Múltiplos asserts em um único teste dificultam o diagnóstico de falhas e escondem quais comportamentos estão sendo verificados. A orientação de um assert por teste não é absoluta, mas serve como critério para avaliar se o teste está focado.

8.2 F.I.R.S.T.

Martin sintetiza as propriedades de bons testes no acrônimo FIRST:

- **Fast** (rápidos): testes lentos não são executados com frequência. Testes unitários devem rodar em milissegundos.
- **Independent** (independentes): testes não devem depender da execução de outros testes. Qualquer teste deve poder ser executado isoladamente.
- **Repeatable** (repetíveis): testes devem produzir o mesmo resultado em qualquer ambiente, sem dependência de banco de dados, rede ou relógio do sistema.
- **Self-Validating** (auto-validáveis): o resultado deve ser binário (passa ou falha), sem necessidade de inspeção manual de logs.
- **Timely** (oportunos): testes devem ser escritos no mesmo momento que o código de produção, ou antes (no caso de TDD).

Exemplo de teste limpo

```
class ContaBancariaTest {

    private ContaBancaria conta;

    @BeforeEach
    void setUp() {
        conta = new ContaBancaria("001");
        conta.depositar(new BigDecimal("500.00"));
    }

    @Test
    void devePermitirSaqueQuandoHaSaldoSuficiente() {
        conta.sacar(new BigDecimal("200.00"));

        assertEquals(new BigDecimal("300.00"), conta.getSaldo());
    }

    @Test
    void deveLancarExcecaoQuandoSaldoInsuficiente() {
        assertThrows(
            SaldoInsuficienteException.class,
            () -> conta.sacar(new BigDecimal("600.00"))
        );
    }

    @Test
```

```

void deveLancarExcecaoParaValorNegativo() {
    assertThrows(
        IllegalArgumentException.class,
        () -> conta.sacar(new BigDecimal("-1.00"))
    );
}
}

```

9. Classes

9.1 Princípio da Responsabilidade Única (SRP)

Uma classe deve ter apenas uma razão para mudar. Uma razão para mudar equivale a uma responsabilidade. Uma classe que gerencia persistência E regras de negócio E formatação de saída tem três razões para mudar: mudança no banco de dados, mudança nas regras e mudança nos requisitos de apresentação. Cada uma dessas razões pode exigir modificação da classe, aumentando o risco de introdução de bugs.

O teste prático é enunciar o propósito da classe em uma frase sem usar "e" ou "ou". Se não for possível, a classe provavelmente tem mais de uma responsabilidade.

9.2 Coesão

Uma classe é coesa quando a maioria dos seus métodos utiliza a maioria de suas variáveis de instância. Baixa coesão indica que a classe pode ser dividida. Quando um grupo de métodos usa apenas um subconjunto das variáveis de instância, isso é um sinal de que existe uma classe menor esperando para ser extraída.

9.3 Abertura para extensão, fechamento para modificação (OCP)

Classes devem ser abertas para extensão e fechadas para modificação. Em termos práticos, adicionar um novo comportamento não deve exigir alterar código existente, mas sim criar novo código (novas subclasses ou implementações de interface). Esse princípio reduz o risco de quebrar funcionalidades já testadas ao introduzir novas.

```

// Violação do OCP: cada novo formato exige
// modificar a classe existente
public class RelatorioExportador {
    public void exportar(Relatorio r, String formato) {
        if (formato.equals("PDF")) { /* ... */ }
        else if (formato.equals("EXCEL")) { /* ... */ }
        // Adicionar CSV exige editar este método
    }
}

```

```

}

// Conforme com OCP: novo formato = nova classe
public interface RelatorioExportador {
    void exportar(Relatorio relatorio);
}

public class ExportadorPdf implements RelatorioExportador {
    public void exportar(Relatorio relatorio) { /* lógica PDF */ }
}

public class ExportadorExcel implements RelatorioExportador {
    public void exportar(Relatorio relatorio) { /* lógica Excel */ }
}

// Adicionar CSV não toca no código existente:
public class ExportadorCsv implements RelatorioExportador {
    public void exportar(Relatorio relatorio) { /* lógica CSV */ }
}

```

10. Princípios complementares

10.1 DRY: Don't Repeat Yourself

Duplicação é a raiz de grande parte do mal em software. Toda vez que um trecho de lógica existe em mais de um lugar, qualquer correção deve ser feita em todos esses lugares, e a probabilidade de esquecer um é alta. A solução é abstrair o trecho duplicado em uma função, classe ou módulo único e referenciá-lo a partir de todos os pontos que precisam daquela lógica.

10.2 KISS: Keep It Simple, Stupid

Sistemas funcionam melhor quando são mantidos simples. A tendência do desenvolvedor experiente é supercomplicar soluções, antecipando requisitos que podem nunca chegar. A regra KISS orienta a resolver o problema presente da forma mais simples possível, sem sacrificar legibilidade.

10.3 YAGNI: You Aren't Gonna Need It

Não implemente funcionalidade antes de precisar dela. Código escrito antecipando necessidades futuras hipotéticas adiciona complexidade real hoje para lidar com problemas imaginários amanhã. Se o requisito não existe hoje, a implementação não deve existir.

10.4 Boy Scout Rule

Deixe o código mais limpo do que você encontrou. A regra não exige refatorações extensas a cada mudança; basta que cada desenvolvedor, ao modificar um arquivo, faça pelo menos um melhoramento pequeno: renomear uma variável, quebrar uma função grande, eliminar um comentário obsoleto. Aplicada consistentemente por uma equipe, essa regra reverte gradualmente a degradação da base de código.

11. Refatoração como prática contínua

Refatoração é o processo de alterar a estrutura interna do código sem modificar seu comportamento externo. É a técnica central para manter o código limpo ao longo do tempo, porque código limpo não é algo que se atinge uma vez: é um estado que precisa ser ativamente preservado contra a pressão de prazos, mudanças de requisito e crescimento da equipe.

A condição para refatorar com segurança é a existência de testes automatizados que garantam que o comportamento não mudou. Sem testes, a refatoração é uma operação de risco que pode introduzir regressões silenciosas.

Refatorações mais comuns em Java

- Renomear variável, método ou classe (Extract/Rename).
- Extrair método: mover um trecho de código para um método nomeado.
- Extrair interface: separar o contrato da implementação.
- Substituir código mágico por constante nomeada.
- Introduzir objeto de parâmetro: agrupar argumentos relacionados em uma classe.
- Substituir condicional por polimorfismo: eliminar cadeias de if/else baseadas em tipo.

Problemático

```
// Código mágico
if (usuario.getPlano() == 2) {
    desconto = 0.15;
}
if (usuario.getPlano() == 3) {
    desconto = 0.25;
}
```

Limpo

```
// Constantes e polimorfismo
public enum PlanoUsuario {
    BASICO(0.0),
    PREMIUM(0.15),
    EMPRESARIAL(0.25);

    private final double desconto;
    // construtor e getter omitidos
}

double desconto = usuario.getPlano()
    .getDesconto();
```

12. Resumo dos princípios

1. Nomes revelam intenção: evite abreviações, nomes de uma letra e termos genéricos.
2. Funções fazem uma coisa: se puder extrair outra função com nome diferente, extraia.
3. Argumento zero é o ideal; booleano como argumento é sinal de que a função deve ser dividida.
4. Comentários compensam código ruim; prefira código expressivo a comentários explicativos.
5. Formatação é comunicação: organize o código para ser lido de cima para baixo.
6. Objetos encapsulam dados e comportamento; estruturas de dados expõem dados sem comportamento.
7. Prefira exceções a códigos de retorno; não retorne nem passe null.
8. Testes limpos seguem FIRST: rápidos, independentes, repetíveis, auto-validáveis e oportunos.
9. Uma classe tem uma razão para mudar (SRP). Coesão alta é sinal de boa separação.
10. DRY, KISS e YAGNI são complementares: elimine duplicação, prefira simplicidade, não antecipe.
11. Refatore continuamente, com cobertura de testes como rede de segurança.

13. Referências

- MARTIN, Robert C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.
- FOWLER, Martin. Refactoring: Improving the Design of Existing Code. 2. ed. Addison-Wesley, 2018.
- HUNT, Andrew; THOMAS, David. The Pragmatic Programmer. 2. ed. Addison-Wesley, 2019.
- GAMMA, Erich et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- Documentação do JUnit 5. Disponível em: <<https://junit.org/junit5/docs/current/user-guide/>>
- Checkstyle: ferramenta de análise estática para Java. Disponível em: <<https://checkstyle.org>>